IMPLIRET: Benchmarking the Implicit Fact Retrieval Challenge

Anonymous ACL submission

Abstract

Retrieval systems are central to many NLP pipelines but often rely on surface-level cues such as keyword overlap and lexical semantic similarity. To evaluate retrieval beyond these shallow signals, recent benchmarks introduce reasoning-heavy queries; however, they primarily shift the burden to query-side processing techniques – like prompting or multi-hop retrieval – that can help resolve complexity. In contrast, we present IMPLIRET, a benchmark that shifts the reasoning challenge to document-side processing: The queries are simple, but relevance depends on facts stated implicitly in documents through temporal (e.g., resolving "two days ago"), arithmetic, and world knowledge relationships. We evaluate a range of sparse and dense retrievers, all of which struggle in this setting: the best nDCG@10 is only 24.87%. We also test whether long-context models can overcome this limitation. But even with a short context of only ten documents, including the positive document, GPT-4.1 scores only 34.06%, showing that document-side reasoning remains a challenge.

1 Introduction

Retrieval systems play a pivotal role in many NLP applications, enabling models to utilize relevant information from large corpora such as document collections, web pages, or conversational histories (Lewis et al., 2020; Gao et al., 2023). Relevance in retrieval can be established through a range of connections, from explicit lexical or semantic similarity to more implicit, context-dependent associations. However, widely used retrieval systems are highly reliant on surface-level cues such as exact matches, repetition, or where a fact appears in the text (Ram et al., 2023; Coelho et al., 2024; Fayyaz et al., 2025). Additionally, many popular benchmarks (e.g., BEIR (Thakur et al., 2021)) do not surface these issues as their queries have lexical overlap with relevant documents (Shao et al., 2025).

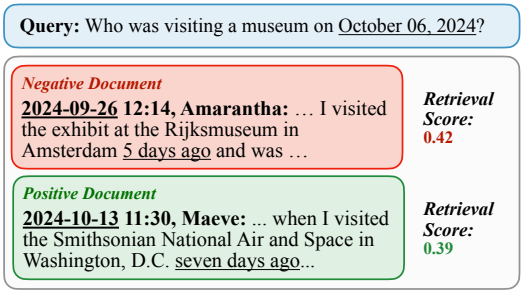

Figure 1: **An example from IMPLIRET:** a query and two sample documents, negative and positive. Retrieval of the relevant positive document requires surfacing implicit knowledge: that Maeve visited the Smithsonian on 2024-10-06.

There are attempts to create reasoning intensive datasets that push beyond lexical and surface-level matches. For instance, RAR-b (Xiao et al., 2024) reframes multiple-choice reasoning tasks into retrieval problems, BIRCO (Wang et al., 2024) collects multi-faceted questions across five domains, and BRIGHT (Su et al., 2025) uses full StackExchange problem descriptions as queries against the pages they cite. Since the reasoning burden lies on the query side, techniques like query expansion, chain-of-retrieval inference, or agentic retrieval can help models handle complex prompts and outperform standard retrievers (Wang et al., 2025; Song et al., 2025; Li et al., 2025).

In contrast, we present **IMPLIRET**, a benchmark that shifts reasoning to document-side processing: the queries are simple, but relevance depends on facts stated implicitly within the documents, spanning **arithmetic**, **temporal**, and **world knowledge** relationships that require inference to uncover. Figure 1 gives an example: the correct document requires resolving a reference to a date that is implicit, i.e., not stated directly. An effective retrieval system must infer such implicit facts from the document content, ideally as part of the indexing process, in order to retrieve the correct result at query

time. Yet current retrieval methods fail to capture the implicit signals needed for accurate retrieval. We evaluate sparse and dense approaches, including BM25 (Robertson and Zaragoza, 2009), ColBERT (Santhanam et al., 2022), and Dragon+ (Lin et al., 2023), and observe consistently poor performance: the best nDCG@10 is only 24.87% across our benchmark. To test whether long-context capabilities could mitigate the problem, we evaluate models in a setting where the positive document is included among several distractors. While GPT-4.1 answers correctly when given only the positive document, its performance drops sharply even with just ten documents in-context, achieving a ROUGE-1 recall of 34.06%.

Our dataset IMPLIRET introduces a new setting that requires document-side reasoning for retrieval rather than query-side reasoning. IMPLIRET presents challenges for both retrieval and long-context processing, highlighting the need for models that can reason over implicit information embedded in large corpora.

2 IMPLIRET

In IMPLIRET, we construct examples whose relevance depends on information that is implicitly stated in the document (i.e., it can only be discovered through reasoning, not by surface-level overlap). IMPLIRET covers three reasoning categories: **World Knowledge**, **Arithmetic**, and **Temporal**.

To generate examples requiring reasoning, we begin with a set of mappings we refer to as **implicit tuple sets**. Each tuple specifies a relationship between an implicit form (used in a document) and its corresponding explicit form (used in the query). Example from Figure 1: ("2024-10-13 ... seven days ago","October 06, 2024").

For each category of reasoning, we define a set of implicit tuples that encode reasoning dependencies. Given a tuple set T, we construct a document pool $\mathcal{D}_T$, where each document corresponds to one tuple $t \in T$, i.e., $|\mathcal{D}_T| = |T|$. The document generated from t serves as the positive document for a query derived from t, while all other documents in $\mathcal{D}_T$ generated from $t' \neq t$ act as negatives for that query. Each tuple set is used to generate examples in either of two distinct styles, *uni-speaker* and *multi-speaker*, with separate pools constructed per style. This separation ensures diversity in surface structure and querying patterns. As a result, every document pool contains exactly one positive sample for a given query and all other samples in the pool serve as semantically irrelevant negatives for that query. In the following, we describe the construction of the implicit tuple sets and the generation process for documents and queries.

2.1 Generating Tuple Sets

Arithmetic. An arithmetic relation requires simple numerical reasoning. For instance, the query "Which bag costs $1,600?" can be answered by "The Prada bag costs $2,000, the Gucci bag is 20% cheaper," since $2,000 × 0.8 = $1600. Here, the model must identify the reference price, interpret the relative statement ("20% cheaper"), and perform the corresponding computation to infer the answer. Therefore, each tuple in the implicit tuple set takes the form $\big((p_1, r, e), p_2\big)$, where p_1 is the base price, r is the relative multiplier, $e \in \{\texttt{"Lower"}, \texttt{"Higher"}\}$ indicates the direction of the change, and p_2 is the queried price (e.g., $((2000, 0.2, \texttt{Lower}), 1600)$). We apply constraints to ensure that queried prices are unique, realistic, and well-distributed across the tuple set. Tuples are generated using a sampling algorithm that selects base prices and checks constraint satisfaction, backtracking as needed until N valid tuples are found (where N is the target number of documents indicated as "Docs" in Table 1). Full constraint details and sampling logic are provided in Appendix 2.1.

World Knowledge. A world knowledge relation connects a textual mention to an external fact. For instance, the query "Who was in the UK?" can be answered by "Lenna was at Big Ben," based on the implicit fact that Big Ben is located in the UK. The model must identify the mentioned entity, retrieve the associated world fact, and use it to resolve the query. Each tuple is encoded as (*landmark*, *country*), e.g., ("Big Ben", "UK"). To build the tuple set, we collect landmark-country pairs that are unambiguous, globally unique, free of lexical cues revealing the country, and refer to specific rather than generic locations. Candidates are sourced from Wikidata (Vrandečić and Krötzsch, 2014) and filtered using LLMs, embedding similarity, and web search verification. Full filtering criteria, prompts, and implementation details are provided in Appendix A.2.

Temporal. A temporal relation involves reasoning over relative dates; we gave an example in Figure 1. The model must identify the reference date (2024-10-13), interpret the relative time expres-

Reasoning	Style	Pools	Docs	Tokens	
				Avg.	Total
Arithmetic	Uni-speaker	50	30	373	559,141
	Multi-speaker	25	20	148	73,879
World-Knowledge	Uni-speaker	50	30	386	579,556
	Multi-speaker	25	20	1423	71,329
Temporal	Uni-speaker	50	30	358	537,144
	Multi-speaker	25	20	131	65,608

Table 1: **IMPLIRET statistics:** number of pools, documents per pool, average document length and total size of the pool.

sion ("seven days ago") and compute the resulting absolute date ("2024-10-06"). Each example is represented as a tuple $\big((d_B, R), D_L\big)$, where d_B is the base date explicitly mentioned in the document, R is a list of relative offsets (e.g. ["1 day after", "2 days after"]), and D_L is the list of resolved explicit dates (e.g., ["March 6th", "March 7th"]). We generate N such tuples under constraints that ensure date uniqueness, broad coverage across a fixed window, and realistic time offsets. Target date sequences are first sampled, then anchored to a base date to define relative expressions. The sampling algorithm verifies constraints and backtracks as needed until a valid set is found. Further details on constraints and sampling logic are provided in Appendix A.3.

2.2 Document-Query Pairs

We generate document-query pairs by converting each fact tuple into a short dialogue or forum post using one of two styles: uni-speaker (multi-turn chat) or multi-speaker (single-post forum).

Uni-speaker (multi-turn chat). For every tuple, we create a document in the form of a short multi-turn dialogue. The same main conversant (e.g., "Alex") appears in every dialogue across the pool. To make the dataset more natural, we vary the second conversant from dialogue to dialogue. Depending on the reasoning category, the main conversant either states which product they bought at a certain price (Arithmetic), mentions visiting a landmark (World Knowledge), or describes an activity that occurred on a specific date (Temporal). The query targets the corresponding implicit fact: the product, person, or activity linked to the given price, country, or date.

Multi-speaker (single-post forum). For each tuple, we create a document in the form of a forum thread where each post in the thread is generated from a tuple. Each post is written in the voice of a different user and all posts responds to a shared prompt. Thus, this style mimics a forum thread in which each user responds independently to a shared prompt. While the actions remain similar to the uni-speaker setting (buying a product, visiting a landmark, or engaging in a scheduled activity), the query perspective shifts. Rather than inquiring about information about a known subject, the query now asks which person or item satisfies a given condition: a price, a location or a dated activity.

Generation Pipeline. In both styles, i.e., in each conversation and post, every message includes a timestamp and speaker name (see Figure 1). While this metadata may not be referenced explicitly in the query, it can be essential for resolving temporal expressions or identifying the correct conversant. In both styles, each example is produced via a three-step pipeline: (1) Entity binding: we assign entities (e.g., names, items, activities) to each tuple to create a plausible scenario and define the query target; (2) Document generation: we prompt an LLM to generate a chat or forum passage that embeds the entity and the implicit part of the tuple, without stating the explicit fact; (3) Verification: a second model attempts to extract the original tuple; we retain only examples where the intended fact is fully recoverable. This pipeline is supported by auxiliary lexical resources, including random names, brand-item pairs, and activity lists, as well as per-reasoning category prompt templates. We use LLAMA 3.3-70B (Meta, 2024) to synthesize the documents for each tuple.[1] Table 1 presents IMPLIRET statistics.

3 Experiments

We use IMPLIRET to test retrieval models on their ability to perform document-side reasoning. Our evaluation covers a wide variety of retrieval methods: sparse lexical baseline BM25 (Robertson and Zaragoza, 2009; Lù, 2024); dense encoders CONTRIEVER, DRAGON+, and REASONIR (Izacard et al., 2021; Lin et al., 2023; Shao et al., 2025); late interaction model COLBERT V2 (Santhanam et al., 2022); and knowledge graph augmented retriever HIPPORAG 2 (Gutiérrez et al., 2025). Each system is tasked with ranking documents to retrieve the relevant passage, with performance measured using nDCG@k[2].

[1] Details such as prompts and query templates are available in Appendix A.4.

[2] We also report MRR@k results in Appendix B.

Retriever	Reasoning			Average
	W. Know.	Arithmetic	Temporal	
Sparse				
BM25	18.97	18.94	16.72	18.21
Late-Interaction				
ColBERT v2	19.47	19.60	17.24	18.77
Dense Encoders				
Contriever	22.84	18.85	19.66	20.45
Dragon+	23.22	19.42	18.50	20.38
ReasonIR	**32.19**	**20.12**	**22.29**	**24.87**
Knowledge-Graph Augmented Indexer				
HippoRAG 2	22.88	19.83	19.77	20.83

Table 2: **Retrieval evaluation. nDCG@10** for our reasoning categories (world knowledge (W. Know.), arithmetic, and temporal, averaged over uni-speaker and multi-speaker documents) and "Average" of reasoning.

Experiment	k	Reasoning			Average
		W. Know.	Arithmetic	Temporal	
Llama 3.3 70B	1	**59.70**	**97.40**	**91.09**	**82.73**
	10	27.35	19.22	21.03	22.53
	All	19.48	6.99	12.49	12.99
GPT-4.1	1	**72.74**	**99.45**	**94.50**	**88.89**
	10	62.17	27.01	13.00	34.06
	All	60.22	14.68	8.41	27.77

Table 3: **RAG-style evaluation. ROUGE-1 (R-1) recall** for our reasoning categories (world knowledge (W. Know.), arithmetic and temporal, averaged over uni-speaker and multi-speaker documents) and "Average" across categories.

3.1 Results

The nDCG@10 results across all reasoning categories are presented in Table 2. The highest average score, 24.87 (achieved by REASONIR, a recent 8B-parameter LLM), shows the difficulty retrieval models face when reasoning over implicit facts in documents. More efficient baselines such as CONTRIEVER, DRAGON+, and BM25 perform substantially worse; notably, BM25 reaches just 18.21 due to its reliance on surface-level lexical overlap.

Performance varies across reasoning types: the World Knowledge category exhibits the largest performance spread (18.97 vs. 32.19), while it is narrowest for Arithmetic (18.94 vs. 20.12). Discourse style also plays a role: REASONIR scores 30.37 on multi-speaker examples compared to 19.36 on uni-speaker ones, suggesting that stylistic structure affects retrieval difficulty.[3]

RAG Evaluation: How much do long-context LLMs help? While retrieval quality can impact performance, a natural question is whether an LLM, particularly one with long-context capacity, can still succeed when the relevant document is present. To test this, we evaluate in a retrieval-augmented generation (RAG) setup where the model is given the question along with k documents: one positive and $k-1$ negatives sampled from the same pool. This isolates the impact of retrieval and focuses purely on document-side reasoning. We evaluate three settings: k=1 (positive only), k=10 (positive plus nine negatives), and a full-pool setting where all documents from the pool are provided as context. The model receives the query along with the sequence of documents and must generate an answer. We evaluate two reader models: LLAMA 3.3 70B and GPT-4.1.[4] In Table 3, we report the average ROUGE-1 recall[5] scores to measure the overlap between the generated output and the positive answer (Lin, 2004). When given only the positive document (k=1), the two models achieve average ROUGE-1 Recall of 82.73 and 88.89. This suggests that the query itself is straightforward to answer once the relevant document is isolated. This also means that an LLM can solve the task if a high-performing retriever (which would retrieve the relevant document at rank 1) is available. However, as k increases (even with the positive included), performance declines, showing that LLMs struggle to focus on the correct evidence amid structurally similar negatives. This supports prior findings on long-context limitations and highlights the need for retrieving a small, focused set of documents rather than increasing context size (Kuratov et al., 2024; Modarressi et al., 2025).

4 Conclusion

We introduce IMPLIRET, a benchmark for evaluating retrieval models when relevance depends on document-side reasoning on implicit facts. Unlike prior datasets that emphasize complex queries, IMPLIRET shifts the reasoning burden to the documents. It covers three reasoning types – world knowledge, arithmetic, and temporal – and two discourse styles. Across sparse, dense, and KG-augmented retrievers, the best nDCG@10 is only 24.87. Even with GPT-4.1, given ten documents including the gold, performance peaks at just 34.06%. These results highlight the difficulty of retrieving implicit facts and the need for models that can reason beyond surface cues.

[3] Full results per category and style in Appendix B.

[4] Checkpoint: `gpt-4.1-2025-04-14`

[5] R-1 Rec. $= \frac{|\text{Output Unigrams} \cap \text{Gold Answer Unigrams}|}{|\text{Gold Answer Unigrams}|}$

Limitations

While our benchmark is carefully designed to evaluate implicit document-side reasoning in retrieval systems, it has the following limitations:

Synthetic Dataset. Documents and queries in IMPLIRET are synthesized using LLMs and structured templates. This allows control over the facts and how they are implicitly expressed, while avoiding conflicts. It also enables easy regeneration if data contamination or memorization is suspected. As with any synthetic benchmark, the data may differ slightly from naturally occurring text in discourse structure or topic diversity. All examples are in English and follow conversational formats (uni-speaker chats and multi-speaker forum posts). Although the use of LLMs helps ensure fluency, it introduces the risk of subtle hallucinations or unintended cues, which we address through automatic verification during dataset construction.

Reasoning Types & Level. In IMPLIRET, we only cover three simple categories of reasoning relations: arithmetic, temporal, and world knowledge, each with shallow composition. While the coverage of reasoning types is limited, the core finding remains: current retrievers struggle to locate relevant documents when reasoning is implicit, and LLMs fail to reliably attend to the correct evidence in long-context settings.

References

João Coelho, Bruno Martins, Joao Magalhaes, Jamie Callan, and Chenyan Xiong. 2024. Dwell in the beginning: How language models embed long documents for dense retrieval. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, pages 370–377, Bangkok, Thailand. Association for Computational Linguistics.

Mohsen Fayyaz, Ali Modarressi, Hinrich Schuetze, and Nanyun Peng. 2025. Collapse of dense retrievers: Short, early, and literal biases outranking factual evidence. *Preprint*, arXiv:2503.05037.

Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, Meng Wang, and Haofen Wang. 2023. Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997*.

Bernal Jiménez Gutiérrez, Yiheng Shu, Weijian Qi, Sizhe Zhou, and Yu Su. 2025. From rag to memory: Non-parametric continual learning for large language models. *arXiv preprint arXiv:2502.14802*.

Gautier Izacard, Mathilde Caron, Lucas Hosseini, Sebastian Riedel, Piotr Bojanowski, Armand Joulin, and Edouard Grave. 2021. Unsupervised dense information retrieval with contrastive learning.

Yuri Kuratov, Aydar Bulatov, Petr Anokhin, Ivan Rodkin, Dmitry Igorevich Sorokin, Artyom Sorokin, and Mikhail Burtsev. 2024. BABILong: Testing the limits of LLMs with long context reasoning-in-a-haystack. In *The Thirty-eight Conference on Neural Information Processing Systems Datasets and Benchmarks Track*.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474. Curran Associates, Inc.

Xiaoxi Li, Guanting Dong, Jiajie Jin, Yuyao Zhang, Yujia Zhou, Yutao Zhu, Peitian Zhang, and Zhicheng Dou. 2025. Search-o1: Agentic search-enhanced large reasoning models. *Preprint*, arXiv:2501.05366.

Chin-Yew Lin. 2004. ROUGE: A package for automatic evaluation of summaries. In *Text Summarization Branches Out*, pages 74–81, Barcelona, Spain. Association for Computational Linguistics.

Sheng-Chieh Lin, Akari Asai, Minghan Li, Barlas Oguz, Jimmy Lin, Yashar Mehdad, Wen-tau Yih, and Xilun Chen. 2023. How to train your dragon: Diverse augmentation towards generalizable dense retrieval. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 6385–6400, Singapore. Association for Computational Linguistics.

Xing Han Lù. 2024. Bm25s: Orders of magnitude faster lexical search via eager sparse scoring. *arXiv preprint arXiv:2407.03618*.

A. Meta. 2024. Llama 3.3 Model Card.

Ali Modarressi, Hanieh Deilamsalehy, Franck Dernoncourt, Trung Bui, Ryan A. Rossi, Seunghyun Yoon, and Hinrich Schütze. 2025. Nolima: Long-context evaluation beyond literal matching. In *Forty-second International Conference on Machine Learning*.

OpenAI. 2024. o1 system card. `https://cdn.openai.com/o1-system-card-20241205.pdf`.

Ori Ram, Liat Bezalel, Adi Zicher, Yonatan Belinkov, Jonathan Berant, and Amir Globerson. 2023. What are you token about? dense retrieval as distributions over the vocabulary. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 2481–2498, Toronto, Canada. Association for Computational Linguistics.

Stephen Robertson and Hugo Zaragoza. 2009. The probabilistic relevance framework: Bm25 and beyond. *Foundations and Trends® in Information Retrieval*, 3(4):333–389.

Keshav Santhanam, Omar Khattab, Jon Saad-Falcon, Christopher Potts, and Matei Zaharia. 2022. ColBERTv2: Effective and efficient retrieval via lightweight late interaction. In *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 3715–3734, Seattle, United States. Association for Computational Linguistics.

Rulin Shao, Rui Qiao, Varsha Kishore, Niklas Muennighoff, Xi Victoria Lin, Daniela Rus, Bryan Kian Hsiang Low, Sewon Min, Wen tau Yih, Pang Wei Koh, and Luke Zettlemoyer. 2025. Reasonir: Training retrievers for reasoning tasks. *Preprint*, arXiv:2504.20595.

Huatong Song, Jinhao Jiang, Yingqian Min, Jie Chen, Zhipeng Chen, Wayne Xin Zhao, Lei Fang, and Ji-Rong Wen. 2025. R1-searcher: Incentivizing the search capability in llms via reinforcement learning. *Preprint*, arXiv:2503.05592.

Hongjin Su, Howard Yen, Mengzhou Xia, Weijia Shi, Niklas Muennighoff, Han yu Wang, Liu Haisu, Quan Shi, Zachary S Siegel, Michael Tang, Ruoxi Sun, Jinsung Yoon, Sercan O Arik, Danqi Chen, and Tao Yu. 2025. BRIGHT: A realistic and challenging benchmark for reasoning-intensive retrieval. In *The Thirteenth International Conference on Learning Representations*.

Nandan Thakur, Nils Reimers, Andreas Rücklé, Abhishek Srivastava, and Iryna Gurevych. 2021. BEIR: A heterogeneous benchmark for zero-shot evaluation of information retrieval models. In *Thirty-fifth Conference on Neural Information Processing Systems Datasets and Benchmarks Track (Round 2)*.

Denny Vrandečić and Markus Krötzsch. 2014. Wikidata: a free collaborative knowledgebase. *Communications of the ACM*, 57(10):78–85.

Liang Wang, Haonan Chen, Nan Yang, Xiaolong Huang, Zhicheng Dou, and Furu Wei. 2025. Chain-of-retrieval augmented generation. *Preprint*, arXiv:2501.14342.

Xiaoyue Wang, Jianyou Wang, Weili Cao, Kaicheng Wang, Ramamohan Paturi, and Leon Bergen. 2024. Birco: A benchmark of information retrieval tasks with complex objectives. *arXiv preprint arXiv:2402.14151*.

Chenghao Xiao, G Thomas Hudson, and Noura Al Moubayed. 2024. Rar-b: Reasoning as retrieval benchmark. *arXiv preprint arXiv:2404.06347*.

A Dataset Generation

In this appendix, we describe, for each of the three reasoning categories, *Arithmetic*, *World-Knowledge*, and *Temporal*, (i) how we construct the implicit-tuple sets and (ii) what arguments are required to synthesize their corresponding contexts. After covering tuple-set construction, we explain in Section A.4 how a language model is used to generate the final passages.

A.1 Arithmetic Reasoning

To generate N implicit reasoning tuples for the Arithmetic category, we use Algorithm 1, which ensures that all tuples satisfy the these constraints: (i) all p_1 and p_2 values across the tuples are distinct, ensuring exactly one correct answer per query; (ii) all p_1 and p_2 values should be a multiple of 10, so that values resemble realistic prices; (iii) the p_2 values are evenly distributed across a predefined range of plausible prices, avoiding value clustering; and (iv) the resulting multiplier must have at most 2 decimals; hence, the required calculation is simple. The Algorithm1 returns a set of tuples of the form $((p_{1,i}, r_i, e_i), p_{2,i})$, where the two prices are mutually distinct and the multiplier r_i satisfies predefined numerical constraints. The construction guarantees uniform distribution across price ranges and ensures that each tuple encodes a plausible relative-price comparison suitable for reasoning-based retrieval.

A.2 World-Knowledge Reasoning

As described in Section 2, we gather landmark-country pairs under three constraints: (i) each landmark must refer to a globally unique location, avoiding names that could correspond to multiple places; (ii) the landmark name must not include lexical, semantic, or language-specific cues that reveal its country, avoiding surface-form shortcuts; and (iii) landmarks must refer to specific, recognizable sites rather than generic ones. To do so, we first assemble a seed list of unique landmark-country pairs via five steps: (i) issue a SPARQL query to Wikidata to retrieve every entity whose `instance-of` (P31) chain includes exactly one of the high-level place classes, museum (`Q33506`), university (`Q3918`), church building (`Q16970`), venue (`Q17350442`), or landmark (the superclass of `Q17350442`), and that is linked to exactly one sovereign state via the `country` (P17) property; (ii) for each hit, extract the English labels

Algorithm 1 Arithmetic Tuple Set

Require: Number of tuples N; *style* $\in$ {multi, uni}; Total number of attempts $limit$

if *style* = multi **then**
  $(B_L, B_U) \leftarrow (50, 2050)$
else
  $(B_L, B_U) \leftarrow (50, 3050)$
end if
$\Delta \leftarrow \lfloor (B_U - B_L)/N \rfloor$
ImplicitTupleSet $\leftarrow \varnothing$
PriceSet $\leftarrow \varnothing$
repeat
  for $i \leftarrow 0$ **to** $N-1$ **do**
    $p_{2,i} \leftarrow B_L + i\Delta$
    $r_i \leftarrow None$
    attempts$\leftarrow 0$
    repeat
      Sample $p_{1,i} \sim \text{Uniform}\big(\{x \in 10\mathbb{N} \mid B_L \le x \le B_U\}\big)$
      if $p_{2,i} > p_{1,i}$ **then**
        $r_i \leftarrow \dfrac{p_{2,i}}{p_{1,i}}$
      else
        $r_i \leftarrow \dfrac{p_{1,i}}{p_{2,i}}$
      end if
      attempts$\leftarrow$*attempts*$+1$
    until $(0 < r_i < 3$ **and** Round(r_i,2) = r_i **and** $p_{1,i} \notin PriceSet$ and $p_{2,i} \notin PriceSet)$ **or** *attempts* exceeds *limit*
    if r_i **then**
      $e_i \leftarrow \begin{cases} \texttt{Lower}, & p_{2,i} < p_{1,i} \\ \texttt{Higher}, & \text{otherwise} \end{cases}$
      ImplicitTupleSet $\leftarrow$ ImplicitTupleSet $\cup\{((p_{1,i}, r_i, e_i), p_{2,i})\}$
    else
      $p_{2,i} \leftarrow p_{2,i} + 1$
      if $p_{2,i} = B_L + (i+1)\Delta$ **then**
        restart entire generation
      end if
    end if
  end for
until |ImplicitTupleSet| $= N$
return ImplicitTupleSet

of its enclosing administrative region (P131), city (P131 restricted to `Q515`), generic location (P276), and street (P669), yielding up to five concentric location strings; (iii) discard entities with missing, machine-generated, or multi-country labels, then drop any whose name tokens overlap these location strings; embed each remaining landmark-country pair with the 768-dimensional Contriever encoder and retain only those with cosine similarity below 0.25; (iv) pass the remainders to a 70B-parameter Llama-3.3 classifier that flags and removes generic names (e.g., "Downtown Club") or labels leaking their country, using exponential back-off retries until accepted; and (v) submit each accepted label to GPT-4o in web-search mode[6], prompting it to return the place's country in a dictionary format, and keep only those for which the model returns exactly one location. This process yields a balanced pool of 100 unique landmark-country pairs, ensuring we sample across different countries rather than multiple landmarks from the same country. Finally, we select a set of distinct countries C and, for each $c \in C$, sample one landmark $l_c \sim \text{Uniform}(L_c)$ from that country's filtered list L_c. The resulting *implicit tuple sets* is as follows:

$$\textit{ImplicitTupleSets} \;=\; \{\,(l_c,\, c) \mid c \in C\,\}.$$

[6]Checkpoint: `gpt-4o-search-preview-2025-03-11`

Algorithm 2 Multi-Speaker Temporal Tuple Set

Require: $DateWindow$ from 2024-01-01 to 2024-12-31
DATESELECTION$(n, DateWindow)$: return n distance *date* in $DateWindow$ in which for $i \in [0, \ldots, n-2]$, days between $date_i$ and $date_{i+1}$ be between 2, up to 7 days, and the distance between $date_{n-1}$ and the end of $DateWindow$ be at least 14 days.

$\{\text{work\_date}_i\}_{i=0}^{19} \leftarrow$ DATESELECTION$(20, DateWindow)$
for $i \leftarrow 0$ **to** 18 **do**
  $\text{message\_date}_i \leftarrow \text{work\_date}_{i+1}$
  $r_i \leftarrow (\text{message\_date}_i - \text{work\_date}_i).\text{days}$
  $TupleSet \leftarrow TupleSet \cup((\text{message\_date}_i, r_i), \text{work\_date}_i)$
end for
$\text{message\_date}_{19} \leftarrow \text{work\_date}_{19} +$ UNIFORMSAMPLE$((1, 7))$
$r_{19} \leftarrow (\text{message\_date}_{19} - \text{work\_date}_{19}).\text{days}$
$TupleSet \leftarrow TupleSet \cup((\text{message\_date}_{19}, r_{19}), \text{work\_date}_{19})$
return $TupleSet$

A.3 Temporal Reasoning

As described in Section 2, we generate a implicit tuple sets of N tuples $\big((d_B, R), D_L\big)$ to have the following constraints: (i) all explicit dates in any D_L are unique within the tuple set, ensuring that each query maps to exactly one positive document; (ii) all dates used as base or resolved targets are evenly distributed across a fixed date window to avoid clustering; and (iii) all relative offsets in R must fall within a limited number of days from. Because context synthesis differs between multi-speaker and uni-speaker modes, we describe the two procedures separately in the following subsections.

A.3.1 Multi-Speaker: Tuple Construction

Here, we generate 25 implicit tuple list, each containing 20 tuples. for generating them,we used Algorithm 2. The returned output contains all the information detailed before.

A.3.2 Uni-Speaker: Tuple Construction

We want to generate 50 implicit tuple sets, each containing 30 tuples. Each tuple set describes a main conversant's 28-day activity schedule. We categorize activities into three types:

1. **One-time:** executed exactly once in the 14-day period (we have 9 for then in each schedule).

2. **Repeating-Non-Sequential:** occurring on multiple, *non-consecutive* days (we have 3 for them in each schedule: 2 days, 3 days, and 2 days).

3. **Repeating-Sequential:** performed on consecutive days (we have 3 for them in each schedule: 3 days, 3 days, and 4 days).

Each set covers two consecutive 14-day blocks, contains exactly 30 activities, and is constructed in three phases: (i) scheduling activities without temporal overlap, (ii) selecting one *message time* per activity such that it differs from the scheduled lot, and (iii) packaging every activity into a tuple (day(s), start_hour, end_hour, message_time). Algorithm 3 details the procedure.

Algorithm 3 Uni–Speaker Temporal Tuple Set

Require: period duration 14-days, day span 7:00–19:00

Auxiliary functions

PLACESCHEDULE(d, T, F, S): place a 2-4 hour scheduled slot, if T = 'Seq', for d consecutive days, if T = 'NonSeq', for d non-consecutive days, and if T = 'Once', for one day; mark blocks in F to remove the free times, add the slot into the schedule list S, and make a tuple from the list of occupied days D_L and start and end hours ((h_{start}, h_{end})) as $a = (D_{La}, h_{start}, h_{end})$ and returns it

SHIFTDATES(S,d_{off}): shift all the relative days to 14 days later if d_{off}=1.

SELECTQTIMES(S): Randomly selects a random day and hour (not exact time of start or end) as question time for each schedule to appear in the query.

DIFFTIMES(m, a): Returns a list of day differences between every scheduled day $d_{i,a}$ in the activity a and m ($(m - d_{i,a}).day$).

$S \leftarrow \varnothing$ ▷ Schedule list
$A \leftarrow \varnothing$ ▷ Activity list
for $period \in \{0, 1\}$ **do** ▷ two consecutive 14-day blocks
  $F \leftarrow \{d \mapsto$INITFREE$(7, 19) \mid d = 1{:}14\}$ ▷ free-time map
  $d_{\text{off}} \leftarrow 14 \cdot period$ ▷ calendar shift
  for all len $\in \{3, 3, 4\}$ **do**
    $A \leftarrow A \cup$PLACESCHEDULE(len, *'Seq'*, F, S)
  end for
  for all len $\in \{2, 2, 3\}$ **do**
    $A \leftarrow A \cup$PLACESCHEDULE(len, *'NonSeq'*, F, S)
  end for
  for $i \leftarrow 1$ **to** 9 **do**
    $A \leftarrow A \cup$PLACESCHEDULE(len, *'Once'*, F, S)
  end for
  SHIFTDATES(S,d_{off})
end for
$\mathcal{Q} \leftarrow$ SELECTQTIMES(S)
TupleSet$\leftarrow \varnothing$
for all activity $a \in A$ **do**
  $m_a \leftarrow$ RANDOM($\mathcal{Q} \setminus \{\text{times}(a)\}$)
  $R_a \leftarrow$ DIFFTIMES(m_a, a) ▷ Message time
  $TupleSet \leftarrow TupleSet \cup ((m_a, R_a), a)$
end for
return *TupleSet*

A.4 Synthesizing the Context

Now, for each reasoning category and document style, we have a list of implicit tuple sets containing all the information needed to have a consistent dataset. Consider having the auxiliary lexical content (personal names, daily-work verbs, brand names with corresponding items, and per-category forum topics and questions needed)[7], to each tuple of the implicit tuple list, we assign unique entities and then each tuple, contains all the information to generating the document in natural way. The exact item required for each reasoning category and style is mentioned in Table 4. Depending on the style, we proceed as follows:

A.4.1 Uni-Speaker (Chat-Style)

In the conversation-generation stage, we load the implicit tuple sets for each (category, style) pair. First, we use the `STARTING_CONVERSATION_PROMPT` to generate 30 unique "starting sentences" for the tuple set, ensuring that no two dialogues begin identically (Figure 2). Next, for each tuple, we prepend one of these starting sentences and feed the combination into the `CONVERSATION_GENERATION_PROMPT`; category-specific prompt templates and requirements are shown in Figure 5 for the Arithmetic, Figure 9 for the World-Knowledge, and Figure 13 for the Temporal reasoning. We then ask the model to produce exactly ten utterances per chat, verify the count, and regenerate any that do not meet this criterion. Finally, each completed conversation is submitted to a separate LLM via the `FEATURE_EXTRACTION_PROMPT`, which must reconstruct the original tuple to confirm that the dialogue faithfully conveys the intended information (Figure 7 for the Arithmetic, Figure 11 for the World-Knowledge, and Figure 15 for the Temporal reasoning).

[7] Generated using GPT-o1 (OpenAI, 2024)

A.4.2 Multi-Speaker (Forum-Style)

In the forum-style generation stage, we similarly load the implicit tuple sets for each (category, style) pair. We first generate 20 unique starting sentences for the tuple set using the `STARTING_CONVERSATION_PROMPT`, so that each reply begins differently (Figure 4 for the Arithmetic, Figure 8 for the World-Knowledge, and Figure 12 for the Temporal reasoning). Then, for each tuple, we provide the forum topic, its base question, one starting sentence, and the tuple data to the `CONVERSATION_GENERATION_PROMPT` configured for forum responses; category-specific templates and requirements again appear in Figure 5 for the Arithmetic, Figure 8 for the World-Knowledge, and Figure 13 for the Temporal reasoning. We generate exactly four sentences per response. Finally, each forum reply is passed to a separate LLM via the `FEATURE_EXTRACTION_PROMPT` to extract the original tuple, ensuring the response accurately encodes the tuple's information (Figure 6 for the Arithmetic, Figure 10 for the World-Knowledge, and Figure 14 for the Temporal reasoning).

B Results

As explained in Section 3.1, our main retrieval metric is nDCG@10. For completeness, we also compute MRR@10, summarized in Table 5. The lower MRR@10 scores confirm that, across reasoning categories, the systems often fail to rank the positive documents first, underscoring the modest overall performance already suggested by nDCG. Granular results for the *Uni-Speaker* and *Multi-Speaker* settings are provided in Table 6. For the RAG-style evaluation, model outputs were generated using the prompt templates shown in Figures 16 and 17, and then evaluated using ROUGE-1 Recall against the reference answer.

Reasoning	Multi-Speaker		Uni-Speaker	
	Each Pool	Each Document	Each Pool	Each Document
Arithmetic	Topic, Forum-Base-Question	Conversant, Brand, Model Years (Two random numbers between 2013 up to 2024, the lower number is the price of the lower-priced item)	Main Conversant	Second Conversant, Shopping item, Low priced brand name, High priced brand
World-Knowledge	Topic, Forum-Base-Question	Conversant, Landmark	Main Conversant	Second Conversant, Landmark, Message Date
Temporal	Topic, Forum-Base-Question	Conversant, Item related to the Forum Base Question,	Main Conversant	Second Conversant, Daily work verb

Table 4: **Entity-assignment granularity for each reasoning category and document style.** Items listed under *Each Pool* are unique in that pool and are never reused across other pools. Items listed under *Each Document* are unique within their own set and never reused across other Documents in that pool. All entities are drawn from the auxiliary lexical resources described in Section A.4.

STARTING_CONVERSATION_PROMPT for Uni-Speaker (Chat Style) documents

Task
Generate {num_starting_points} distinct, natural-sounding first sentences suitable as the opening line of a conversation between two friends.

Requirements
- No numbering, bullets, or extra text before or after each sentence.
- Tone must be friendly, approachable, and universally applicable.
- Avoid any numerical mentions.
- Do not mention purchases or someone buying something.
- Do not include temporal or numerical references.
- Avoid common starting phrases; strive for originality.

Output Format
At least {num_starting_points} distinct sentences.
Separate each sentence with a blank line.

Figure 2: Prompt for generating a list of first sentences in Uni-Speaker (Chat Style) documents. This prompt is used for all the reasoning categories of the Arithmetic, World-Knowledge, and Temporal.

Retriever	Reasoning			Average
	W. Know.	Arithmetic	Temporal	
Sparse Baseline				
BM25	12.23	12.19	10.28	11.57
Late-Interaction				
ColBERT v2	12.44	12.89	10.84	12.06
Dense Encoders				
Contriever	15.25	12.20	12.65	13.37
Dragon+	16.04	12.74	11.71	13.49
ReasonIR	**23.96**	**13.33**	**14.75**	**17.35**
Knowledge-Graph–Augmented Indexer				
HippoRAG 2	15.24	13.17	12.94	13.78

Table 5: **MRR@10** ranking metric scores for our reasoning category of World-Knowledge (W. Know.), Arithmetic, and Temporal, averaged over both Uni-Speaker and Multi-Speaker documents. The final "Average" column reports the mean MRR@10 across all reasoning categories.

Experiment	World-Knowledge				Arithmetic				Temporal				Average			
	Uni-Speaker		Multi-Speaker		Uni-Speaker		Multi-Speaker		Uni-Speaker		Multi-Speaker		Uni-Speaker		Multi-Speaker	
	MRR@10	nDCG@10	MRR@10	nDCG@10	MRR@10	nDCG@10	MRR@10	nDCG@10	MRR@10	nDCG@10	MRR@10	nDCG@10	MRR@10	nDCG@10	MRR@10	nDCG@10
Sparse Baseline																
BM25	9.80	15.21	14.66	22.72	9.67	15.11	14.71	22.77	8.05	13.08	12.51	20.37	9.18	14.47	13.96	21.96
Late-Interaction																
ColBERT v2	9.82	15.47	15.07	23.48	**10.71**	**16.32**	15.08	22.87	8.37	13.45	13.31	21.04	9.63	15.08	14.48	22.46
Dense Encoders																
Contriever	11.17	17.01	19.32	28.66	9.92	15.25	14.49	22.46	9.35	14.75	15.95	24.57	10.15	15.67	16.59	25.23
Dragon+	11.73	17.55	20.35	28.88	9.79	15.14	15.68	23.69	8.79	14.06	14.62	22.94	10.10	15.58	16.88	25.17
ReasonIR	**19.35**	**26.70**	**28.56**	**37.67**	10.08	15.47	**16.58**	**24.78**	**10.23**	**15.92**	**19.27**	**28.65**	**13.22**	**19.36**	**21.47**	**30.37**
Knowledge-Graph–Augmented Indexer																
HippoRAG 2	11.47	17.39	19.01	28.37	10.04	15.37	16.30	24.29	9.48	14.71	16.39	24.82	10.33	15.82	17.24	25.83

Table 6: **MRR**@10 and **nDCG**@10 ranking metric scores for each reasoning category of World-Knowledge, Arithmetic, and Temporal, separated for each of Uni-Speaker and Multi-Speaker documents. The final "Average" column reports the mean MRR@10 and nDCG@10 across all categories.

STARTING_CONVERSATION_PROMPT for Multi-Speaker (Forum Style) documents

Task
Generate {num_starting_points} distinct, natural-sounding first sentences suitable as the opening line of a response in an online forum discussion.

Requirements
- No numbering, bullets, or extra text before or after each sentence.
- Tone must be friendly, approachable, and universally applicable.
- Avoid any topic-specific references.
- Use general phrasing.
- Do not mention purchases or someone buying something.
- Do not include numerical references in the sentences.

Output Format
At least {num_starting_points} distinct sentences.
Separate each sentence with a blank line.

Figure 3: Prompt for generating a list of first sentences in Multi-Speaker (Forum Style) documents. This prompt is used for all the reasoning categories of the Arithmetic, World-Knowledge, and Temporal.

CONVERSATION_GENERATION_PROMPT for Arithmetic Reasoning, Multi-Speaker (Forum Style) documents

Task
Generate a natural response to a forum question.

Input
- `topic`: A short topic of the forum discussion.
- `forum_question`: A base question posted in the forum.
- `forum_post`: A list of three sentences:
  1. The price of an item from a certain brand's model to dollars (e.g., "Gaming Chairs from Secretlab, model 2019: 1650").
  2. The price of another item from the same brand but a different model, the price is described in relative form to the first item (e.g., "Secretlab, model 2019: 2.5 times more expensive than model 2016").
  3. A sentence stating which model of the brand was ultimately purchased (e.g., "model 2016 was purchased").
- `first_sentence`: The first sentence of the response.

Requirements
- In the response, you should answer the "forum_question" by using the information in "forum_post".
- You can use the information in "first_sentence" (modify it if needed) to start the conversation.
- Preserve the numeric references (prices, multipliers, etc.).
- You can write the numbers as words, but do not change the value at all (e.g., 3.5 can be mentioned as "three and a half").
- Write the relative price in a natural way (e.g., "Secretlab, model 2019: 2.5 times more expensive than model 2016" can be mentioned as "The Secretlab 2019 model costs two and a half times as much as the 2016 model.").
- Explicitly mention the brand and model references, or the model was purchased.
- Only mention the information once in the response.
- Make sure that you generate grammatically correct sentences.
- Optionally include a reason for choosing the second brand.
- Your generated answer must be coherent and make the Answer natural as a human is really answering the `forum_question` in 4 sentences.

Output Format
Only 1 line of response, without any prefix or suffix.

INPUT: {context}

Figure 4: Prompt for generating the conversations for Multi-Speaker (Forum Style) documents in the Arithmetic reasoning.

CONVERSATION_GENERATION_PROMPT for Arithmetic Reasoning, Uni-Speaker (Chat Style) documents

Task
Generate a natural conversation between two people ("user_1" and "user_2") based on a shopping list.

Input
- `user_1`: Name of the first user.
- `user_2`: Name of the second user.
- `shopping_type`: Type of shopping.
- `item_to_buy`: The purchased item.
- `bought`: A list of three sentences:
  1. The price of the item in another brand.
  2. The price of the item in the brand bought, relative to the first.
  3. The brand bought.
- `first_sentence`: The first sentence of the conversation.

Requirements
- In the conversation, "user_1" will share a shopping information message and should mention that wit as in the 'shopping_type' category and bought the 'item_to_buy', while "user_2" must engage naturally but must not reveal or comment on any shopping or locational information.
- You can use the information in "first_sentence" (modify it if needed) to start the conversation.
- Preserve exact numbers and relative phrasing in the `bought` sentences.
- Explicitly state that "user_1" did not buy from the first brand.
- Explicitly state that "user_1" bought from the second brand.
- "user_2" replies naturally without referencing shopping or numerical details.
- Make sure that you generate grammatically correct sentences.
- Mention exact brands and shopping types as given once in the conversation.
- Optionally include a reason for choosing the second brand.
- The conversation must consist of exactly 10 utterances.
- Each utterance is on its own line.

Output Format
Only 10 lines of dialogue are separated by newlines. For each line, separate the user name (one of the values of `user_1` or `user_2`) and the utterance with a colon

INPUT: {context}

Figure 5: Prompt for generating the conversations for Uni-Speaker (Chat Style) documents in the Arithmetic reasoning.

FEATURE_EXTRACTION_PROMPT for Arithmetic Reasoning, Multi-Speaker (Forum Style) documents

Task
Identify purchased items, their brands, and prices in a conversation transcript.

Input
You are given a forum response transcript with 4 sentences as INPUT.

Requirements
- Your task is to identify items purchased in the conversation, the brand of those items, and their prices. Note that the cost might be stated explicitly or described in relative terms. In such relative cases, you must calculate the final price numerically based on the information available.
- Identify any items that someone actually buys or mentions buying.
- Determine the brand associated with each purchased item (if specified).
- Extract or compute the price in dollars, performing calculations for relative pricing.
- If no purchases are found, return an empty list.
- Do not include any additional commentary.

Output Format
A list of dictionaries with keys:
- `item` (string): Name of the purchased item.
- `brand` (string): Brand name.
- `model` (integer): Model of the item.
- `price` (integer): Price in dollars.

INPUT: {context}

Figure 6: Prompt for reconstructing the original tuple of implicit tuple set (extracting features) from generated conversations for Multi-Speaker (Forum Style) documents in Arithmetic reasoning.

FEATURE_EXTRACTION_PROMPT for Arithmetic Reasoning, Uni-Speaker (Chat Style) documents

Task
Identify purchased items, their brands, and prices in a conversation transcript.

Input
Input is a conversation transcript as a list of lines, each:
<user name>: <utterance>

Requirements
- Detect items someone buys or mentions buying.
- Determine the brand for each purchased item.
- Extract or compute the price in dollars, performing calculations for relative pricing.
- If no purchases are found, return an empty list.
- Do not include any additional commentary.

Output Format
A list of dictionaries with keys:
- `item` (string): Name of the purchased item.
- `brand` (string): Brand name.
- `price` (integer): Price in dollars.

INPUT: {context}

Figure 7: Prompt for reconstructing the original tuple of implicit tuple set (extracting features) from generated conversations for Multi-Speaker (Forum Style) documents in Arithmetic reasoning.

CONVERSATION_GENERATION_PROMPT for World-Knowledge Reasoning, Multi-Speaker (Forum Style) documents

Task
Generate a natural response to a forum question.

Input
- `topic`: A short topic of the forum discussion.
- `forum_question`: A base question posted in the forum.
- `forum_post`: The location that the person wants to use for responding to the "forum_question".
- `type_of_location`: The type of location that the person in the post is talking about in "forum_post".
- `first_sentence`: The first sentence of the response.

Requirements
- In the response, you should answer the "forum_question" by using the information in "forum_post".
- You can use the information in "first_sentence" (modify it if needed) to start the conversation.
- You should mention that the user was in the location mentioned in "forum_post" and you can use the information in "type_of_location" to know the type of location mentioned in "forum_post".
- Make sure that you generate grammatically correct sentences.
- Only mention the `forum_post` information once in the response.
- Do not alter the location in `forum_post`, use it exactly as it is without any changes.
- Do not mention any other location than the one in `forum_post`
- Optionally include a reason for choosing the second brand.
- Your generated answer must be coherent and make the Answer natural as a human is really answering the `forum_question` in 4 sentences.

Output Format
Only 1 line of response, without any prefix or suffix.

INPUT: {context}

Figure 8: Prompt for generating the conversations for Multi-Speaker (Forum Style) documents in the World-Knowledge reasoning.

CONVERSATION_GENERATION_PROMPT for World-Knowledge Reasoning, Uni-Speaker (Chat Style) documents

Task
Generate a natural conversation between two people ("user" and "user_2") based on a trip.

Input
- `user_1`: Name of the first user.
- `user_2`: Name of the second user.
- `trip_destination`: Destination of the trip.
- `type_of_location`: Type of location.
- `trip_friends`: Friends of the trip.
- `first_sentence`: The first sentence of the conversation.

Requirements
- In the conversation, "user_1" will share a trip information message and should mention that it was in the 'trip_destination' with the 'trip_friends', while "user_2" must engage naturally but must not reveal or comment on any trip or locational information.
- You can use the information in "first_sentence" (modify it if needed) to start the conversation.
- "user_2" replies naturally without referencing locational information.
- Do not mention any other locational information in the conversation. Do not mention the country or city of the trip destination.
- Make sure that you have exactly mentioned the 'trip_destination' in the conversation without any other information or changes.
- Mention the 'trip_destination' exactly as it is once in the conversation.
- Make sure that you generate grammatically correct sentences.
- Do not mention the country or city of the 'trip_destination'.
- "type_of_location" is the type of location that the "user_1" has gone to, which can help you make the conversation more natural.
- The conversation must consist of exactly 10 utterances.
- Each utterance is on its own line.

Output Format
Only 10 lines of dialogue are separated by newlines. For each line, separate the user name (one of the values of `user_1` or `user_2`) and the utterance with a colon

INPUT: {context}

Figure 9: Prompt for generating the conversations for Uni-Speaker (Chat Style) documents in the World-Knowledge reasoning.

FEATURE_EXTRACTION_PROMPT for World-Knowledge Reasoning, Multi-Speaker (Forum Style) documents

Task
Identify the location that the person was in.

Input
You are given a forum response transcript with 4 sentences as INPUT.

Requirements
- Your task is to identify the location that the person was in.
- Extract the exact name and do not change it.
- If no location is found, return an empty list.
- Do not include any additional commentary.

Output Format
A list of dictionaries with the key:
- `location` (string): Name of the location that the person was in.

INPUT: {context}

Figure 10: Prompt for reconstructing the original tuple of implicit tuple set (extracting features) from generated conversations for Multi-Speaker (Forum Style) documents in the World-Knowledge reasoning.

FEATURE_EXTRACTION_PROMPT for World-Knowledge Reasoning, Uni-Speaker (Chat Style) documents

Task
Identify places that the user has gone or visited, and the people that the user has gone with.

Input
Input is a conversation transcript as a list of lines, each:
<user name>: <utterance>

Requirements
- Detect places that the user has gone to or visited.
- Determine the people that the user has gone with.
- If no places or people are found, return an empty list.
- Do not include any additional commentary.

Output Format
A list of dictionaries with keys:
- `destination` (string): Name of the place that the user has gone or visited.
- `friends` (string): Name of the people that the user has gone with.

INPUT: {context}

Figure 11: Prompt for reconstructing the original tuple of implicit tuple set (extracting features) from generated conversations for Uni-Speaker (Chat Style) documents in the World-Knowledge reasoning.

CONVERSATION_GENERATION_PROMPT for Temporal Reasoning, Multi-Speaker (Forum Style) documents

Task
Generate a natural response to a forum question.

Input
- `topic`: A short topic of the forum discussion.
- `forum_question`: A base question posted in the forum.
- `forum_post`: The item related to the topic that the person wants to use for responding to the "forum_question".
- `offset_days`: The date that the person in the post is talking about in "forum_post".
- `first_sentence`: The first sentence of the response.

Requirements
- In the response, you should answer the "forum_question" by using the item mentioned in "forum_post".
- You can use the information in "first_sentence" (modify it if needed) to start the conversation.
- You should mention that the user had done the work on the date mentioned in "offset_days", based on the 'topic', choose a verb that is suitable for the work.
- The work should be done exactly in one day, so avoid using vague temporal references like "until", "by the ...", "completed", "finished", etc.
- Do not alter the `offset_days`, use it exactly as it is without any changes, only you are allowed to say it with words or numbers (e.g., "2 days ago", "two days ago").
- Do not mention any other item than the one in "forum_post".
- Do not mention any date other than the one in "offset_days".
- Only mention the `forum_post` information once in the response.
- Make sure that you generate grammatically correct sentences.
- Your generated answer must be coherent and make the Answer natural as a human is really answering the `forum_question` in 4 sentences.

Output Format
Only 1 line of response, without any prefix or suffix.

INPUT: {context}

Figure 12: Prompt for generating the conversations for Multi-Speaker (Forum Style) documents in the Temporal reasoning.

CONVERSATION_GENERATION_PROMPT for Temporal Reasoning, Uni-Speaker (Chat Style) documents

Task
Generate a natural conversation between two people ("user_1" and "user_2") based on the given schedule.

Input
- `user_1`: Name of the first user.
- `user_2`: Name of the second user.
- `work`: The work task.
- `activity_type`: the repeating type of the activity, it can be "One-Time" (that means the user have to do the "work" in the day dated and mentioned "hour"), "Repeating-Sequential" (that means we have a sequential days that the user have to do the "work" in mentioned "hour"), "Repeating-Non-Sequential" (that means the "work" will be done in different days that maybe are not in following days)
- `days`: the dates on which the work has been done.
- `hours`: The hours that the work is to be performed.
- `offset_days`: The list of offset_days of the work, if it is a list of one item, it means the work was done on that day, if it is a list of more than one items, it means the work was done on multiple days.
- `message_time`: The time that the conversation is being sent: [date of the message, day of the week, hour in 24h format]
- `first_sentence`: The first sentence of the conversation.

Requirements
- In the conversation, "user_1" will share a schedule information message and should mention that they did the 'work' on a day or days and 'hours', while "user_2" must engage naturally but must not reveal or comment on their schedules.
- You can use the information in "first_sentence" (modify it if needed) to start the conversation.
- You should mention that the 'user_1' did the 'work' on the specific day or days. Mention the day of working relative to today with the help of the corresponding 'offset_days'. For each item of 'offset_days', if the item is negative, it means the work was done before today (e.g, -5 means "5 days ago"); if it is positive, it means the work was done after today (e.g, 4 means "4 days later").
- Make sure that you mentioned all the days in the 'offset_days' list.
- You can also mention the days relative to each other in the conversation, for example, if the 'offset_days' is [2, 6, 12], you can say it in this way: "2 days later, 4 days after that, and 6 days after the second day".
  For example, if the work day is ['2020-01-12'] and the message day is ['2020-01-13']. Here `offset_days` is [-1] : (2020-01-12) - (2020-01-13) = -1. Hence, you can say "yesterday".
  Or if the message_date is ['2021-02-15'] and work days are ['2021-02-17', '2021-02-18', '2021-02-19']. Here `offset_days` are [2, 3, 4]. Hence, you can say "from two days later, for 3 consecutive days".
  Or if the message_date is ['2021-03-11'] and work days are ['2021-03-16', '2021-03-18'] and the "hours" is (11,15). Here `offset_days` is [5, 7], and the difference is 7-5=2. Hence, you can say "5 days later, and two days after that, from 11 in the morning for 4 hours on both days" or "5 days later, and two days after that, from 11 a.m. for 4 hours on both days"
  Or if the message_date is ['2022-11-10'] and work days are ['2021-11-16', '2021-11-19', '2021-11-23'] and the "hours" is (10,12). Here `offset_days` is [6, 9, 13], and the difference is 9-6=3, and 13-9=4. Hence, you can say "6 days later, and 3 days after that, and 4 days after the second day from 11 in the morning for 4 hours on both days" or "5 days later, and to days after that, from 11 a.m. for 4 hours on both days"

- Do not mention any explicit date for working, only mention the days relative to today.
- All the work are being done in the same hour mentioned in 'hours', you should not directly mention the end hour, but make sure that you accurately mention end hour relative to the start hour (e.g., "from 1 p.m. until 3 hours after that" or "from 9 in the morning for three hours").
- Mention the `work` in the conversation exactly as it is (only change the tense if needed).
- Do not change the date information. Ensure that the "hours" you use in the conversation for "work" are correct and accurate. For example, if the work is "updating a work log" and the "message_time" is ("2023-07-21", "Friday", 14), and the "hours" are (7, 10), You can use it like this: "2023-07-21", "Alaina", "I have to update a work log tomorrow from 7 in the morning for three hours."
- The message time is the time at which the conversation is being sent; you can only choose a random minute for the conversation. The format should be like this: "YYYY-MM-DD HH:MM" (e.g., "2024-01-01 12:00").
- "user_2" replies naturally without referencing schedule or numerical details.
- Mention the working schedule once in the conversation.
- Make sure that you generate grammatically correct sentences.
- The conversation must consist of exactly 10 utterances.
- Each utterance is on its own line.

Output Format
Only 10 lines of dialogue are separated by newlines. For each line, separate the message time and the user name (one of the values of `user_1` or `user_2`) with a comma, and separate the user name and the utterance with a colon.

INPUT: {context}

Figure 13: Prompt for generating the conversations for Uni-Speaker (Chat Style) documents in the Temporal reasoning.

FEATURE_EXTRACTION_PROMPT for Temporal Reasoning, Multi-Speaker (Forum Style) documents

Task
Identify a work-related task described in the user's mention for the forum response and extract its temporal details. Specifically, you should:
1. Determine the work task (e.g., the action or project mentioned).
2. Identify any temporal expressions referring to when the work is to be performed. Convert relative time expressions (such as "tomorrow", "next week", etc.) into numerical offset_days (e.g., "1 day ago", "2 days ago", "3 days ago", etc.). Be very careful that the relevant dates are correct.

Input
You are given a forum response transcript with 4 sentences as INPUT.

Requirements
- Your task is to identify the work task and the `offset_days`.
- Mention the `offset_days` as a number with words (e.g., "1 day ago", "2 days ago", "3 days ago", etc.).
- Extract the exact work task and do not change it.
- If no work task or offset_days is found, return an empty list.
- Do not include any additional commentary.

Output Format
A list of dictionaries with the keys:
- `work` (string): The work task.
- `days` (string): The offset_days.

INPUT: {context}

Figure 14: Prompt for reconstructing the original tuple of implicit tuple set (extracting features) from generated conversations for Multi-Speaker (Forum Style) documents in the Temporal reasoning.

FEATURE_EXTRACTION_PROMPT for Arithmetic Reasoning, Uni-Speaker (Chat Style) documents

Task
Identify a work-related task described in the conversation and extract its temporal details.

Input
Input is a conversation transcript as a list of lines, each:
<message time>, <user name>: <utterance>

Requirements
- Determine the work task (e.g., the action or project mentioned).
- Identify any temporal expressions referring to when the work is to be performed. Convert relative time expressions (such as "tomorrow", "next week", etc.) into absolute dates (YYYY-MM-DD) using the conversation date as a reference. Be very careful that the relevant dates be correct.
- Extract the time range mentioned for the task and express it as a tuple of two integers representing the start and end hours in 24-hour format.
- If no work task or offset_days is found, return an empty list.

Output Format
A list of dictionaries with keys:
- `work` (string): A string describing the identified task.
- `days` (list): A list of one or more dates (YYYY-MM-DD) on which the task occurs.
- `hours` (tuple): A tuple of two integers representing the start and end hours.

INPUT: {context}

Figure 15: Prompt for reconstructing the original tuple of implicit tuple set (extracting features) from generated conversations for Uni-Speaker (Chat Style) documents in the Temporal reasoning.

RAG_STYLE_PROMPT for Multi-Speaker (Forum Style) documents

Task
Answer the Question based on the context provided.

Input
- The INPUT contains a Topic, Forum Question and several Responses to the Forum Question.
- Each response is mentioned by a number in the following format:
Response {{number}}:
- Each response is separated by a new line.
- Each response contains the date, speaker and the message in the following format:
<date>, <speaker>: <message>

Output
return the final answer in a new line after "Answer:" without any prefix or suffix.

INPUT:
{context}

Answer the following question as precisely as possible, using the information provided in the responses. You may rely on the response content, the time each response was sent, and who sent it.

Question: {question}

Answer:

Figure 16: Prompt for RAG-style experiment, while the input is forced to contain the positive document, in Multi-Speaker (Forum Style). This prompt is used for all the reasoning categories of the Arithmetic, World-Knowledge, and Temporal.

RAG_STYLE_PROMPT for Uni-Speaker (Chat Style) documents

```
**Task**
Answer the Question based on the context provided.

**Input**
- The INPUT contains several conversations between two users as Context and a Question.
- Each conversation is mentioned by a number in the following format:
Conversation {{number}}:
- Each conversation contains 10 utterances that are separated by lines.
- Each utterance contains the date, speaker and the message in the following format:
<date>, <speaker>: <message>

**Output**
return the final answer in a new line after "Answer:" without any prefix or suffix.

INPUT:
Context: {context}

Answer the following question as precisely as possible, using the information provided in the conversation. You may
rely on the conversation content, the time each conversation was sent, and who sent it.

Question: {question}

Answer:
```

Figure 17: Prompt for RAG-style experiment, while the input is forced to contain the positive document, in Uni-Speaker (Chat Style). This prompt is used for all the reasoning categories of the Arithmetic, World-Knowledge, and Temporal.